ESTÁNDARES Y NORMATIVAS DE TESTING

Marco Normativo Aplicado al Proceso de Pruebas

Módulo: Actividades Complementarias — Odoo 19 Community

Proyecto	Sistema de Gestión de Actividades Complementarias
Sistema evaluado	actividades_complementarias — Odoo 19 Community
Fecha	22 de Marzo 2026
Normativas cubiertas	ISO/IEC/IEEE 29119 (Partes 1-5) | ISO/IEC 25010 | OCA Guidelines | Odoo SA Guidelines

Introducción

El presente documento describe el marco normativo y los estándares internacionales que fundamentan el proceso de pruebas aplicado al módulo de Actividades Complementarias sobre Odoo 19 Community. Cada norma se presenta de forma individual con su definición oficial y una explicación de cómo se aplica concretamente en este proyecto.

Las normativas se dividen en dos categorías: estándares internacionales de calidad de software (ISO/IEC/IEEE) y directrices propias del ecosistema Odoo (OCA y Odoo SA).

1. ISO/IEC/IEEE 29119 — Pruebas de Software

La norma ISO/IEC/IEEE 29119 es el estándar internacional más completo dedicado exclusivamente a las pruebas de software. Define terminología, procesos, documentación y técnicas de diseño aplicables a cualquier ciclo de vida de desarrollo.

ISO/IEC/IEEE 29119-1 ISO / IEC / IEEE 2013 — rev. 2022	**Conceptos y Definiciones**

Definición oficial

> *This part of ISO/IEC/IEEE 29119 provides a vocabulary of terms used in software testing that are applicable to all subsequent parts of this standard. It also defines the concepts that underlie the software testing process described in the other parts.*
>
> **— ISO/IEC/IEEE 29119-1:2022, Cláusula 1**

Traducción

> *Esta parte de la ISO/IEC/IEEE 29119 proporciona un vocabulario de términos utilizados en las pruebas de software que son aplicables a todas las partes posteriores de este estándar. También define los conceptos que sustentan el proceso de pruebas de software descrito en las demás partes.*
>
> **— ISO/IEC/IEEE 29119-1:2022, Cláusula 1**

La Parte 1 establece el vocabulario y los conceptos base que unifican el lenguaje del proceso de pruebas: qué se entiende por 'defecto', 'fallo', 'caso de prueba', 'cobertura' y 'criterio de salida', entre otros. Sin esta base común, equipos distintos usarían los mismos términos con significados diferentes.

App

Aplicación en este proyecto

En base a la norma ISO/IEC/IEEE 29119-1, los defectos documentados en este proyecto se clasifican con terminología oficial: se distingue entre 'defecto' (causa raíz en el código), 'fallo' (comportamiento incorrecto observable por el usuario) y 'error' (equivocación durante el desarrollo). Por ejemplo, el D-04 (rango de créditos incompleto) es un defecto en el modelo que produce un fallo visible cuando el Comité intenta asignar 1.5 o 2 créditos y no encuentra la opción.

ISO/IEC/IEEE 29119-2 ISO / IEC / IEEE 2021	**Procesos de Prueba**

Definición oficial

> *This document establishes a framework for software testing processes that can be used within any software development lifecycle. It defines test management processes and dynamic test processes: test planning, test monitoring and control, test design and implementation, test environment setup and maintenance, test execution, test incident reporting, and test completion.*
>
> **— ISO/IEC/IEEE 29119-2:2021, Cláusula 1**

Traducción

> *Este documento establece un marco para los procesos de pruebas de software que puede utilizarse dentro de cualquier ciclo de vida de desarrollo de software. Define procesos de gestión de pruebas y procesos de pruebas dinámicas: planificación de pruebas, monitoreo y control, diseño e implementación de pruebas, configuración y mantenimiento del entorno de pruebas, ejecución de pruebas, reporte de incidentes y finalización de pruebas.*
>
> **— ISO/IEC/IEEE 29119-2:2021, Cláusula 1**

La Parte 2 define los procesos que debe seguir cualquier actividad de pruebas: desde la planificación inicial hasta el cierre del ciclo. Establece que las pruebas deben tener criterios de entrada (condiciones para comenzar), criterios de salida (condiciones para concluir) y un plan documentado que defina alcance, riesgos y estrategia.

App	**Aplicación en este proyecto** En base a la norma ISO/IEC/IEEE 29119-2, este proyecto definió criterios de entrada (módulo instalado sin errores, datos de prueba cargados, usuarios de cada grupo creados) y criterios de salida (0 defectos críticos abiertos, cobertura ≥ 80%, UAT firmado). El plan de pruebas identifica los módulos JD-01SC al CA-02SC y prioriza los riesgos más altos como la ausencia de ir.rule y el rango incompleto de créditos.

ISO/IEC/IEEE 29119-3 ISO / IEC / IEEE 2021	**Documentación de Pruebas**

Definición oficial

> *This document specifies the content of test documentation, including test plans, test design specifications, test case specifications, test procedure specifications, test data requirements, test data readiness reports, test execution logs, test incident reports, and test reports.*
>
> **— ISO/IEC/IEEE 29119-3:2021, Cláusula 1**

Traducción

> *Este documento especifica el contenido de la documentación de pruebas, incluyendo planes de prueba, especificaciones de diseño de pruebas, especificaciones de casos de prueba, especificaciones de procedimientos de prueba, requisitos de datos de prueba, reportes de preparación de datos, registros de ejecución, reportes de incidentes y reportes de pruebas.*
>
> **— ISO/IEC/IEEE 29119-3:2021, Cláusula 1**

La Parte 3 especifica exactamente qué documentos deben producirse durante las pruebas y qué información debe contener cada uno. Establece que cada defecto debe registrarse con su ID, descripción, severidad, módulo afectado y pasos para reproducirlo, formando un 'Test Incident Report'.

App	**Aplicación en este proyecto** En base a la norma ISO/IEC/IEEE 29119-3, el informe de defectos de este proyecto sigue la estructura de 'Test Incident Report': cada defecto (D-00 a D-19) incluye identificador único, caso de uso relacionado, módulo y archivo afectado, regla de negocio violada, comportamiento observado y corrección sugerida. Los casos de prueba en formato Gherkin corresponden a la 'Test Case Specification'.

ISO/IEC/IEEE 29119-4 ISO / IEC / IEEE 2021	**Técnicas de Diseño de Pruebas**

Definición oficial

> *This document defines software test design techniques that can be used to derive or select test cases for dynamic testing. It covers black-box techniques (equivalence partitioning, boundary value analysis, decision table testing, state transition testing, use case testing) and white-box techniques (statement testing, branch testing, MC/DC).*
> **— ISO/IEC/IEEE 29119-4:2021, Cláusula 1**

Traducción

> *Este documento define técnicas de diseño de pruebas de software que pueden utilizarse para derivar o seleccionar casos de prueba para pruebas dinámicas. Cubre técnicas de caja negra (particiones de equivalencia, análisis de valores límite, tablas de decisión, transiciones de estado, pruebas de casos de uso) y técnicas de caja blanca (pruebas de sentencias, de ramas y MC/DC).*
> **— ISO/IEC/IEEE 29119-4:2021, Cláusula 1**

La Parte 4 define las técnicas concretas para diseñar casos de prueba. Las técnicas de caja negra se basan en las especificaciones sin conocer el código (equivalencia de particiones, valores límite). Las técnicas de caja blanca requieren conocer la implementación y miden cobertura de sentencias y ramas.

App	**Aplicación en este proyecto** En base a la norma ISO/IEC/IEEE 29119-4, este proyecto aplicó análisis de valores límite en el campo de créditos (0.5, 1.0, 1.5, 2.0 — identificando que 1.5 y 2.0 son valores límite faltantes, defecto D-04), particiones de equivalencia en los estados de actividad (válidos: en_revision, aprobada, rechazada, pendiente_inicio, en_curso, finalizada; inválidos: vacío) y cobertura de ramas con coverage.py en action_aprobar() y action_rechazar().

ISO/IEC/IEEE 29119-5 ISO / IEC / IEEE 2016	**Pruebas Basadas en Palabras Clave**

Definición oficial

> *This part of ISO/IEC/IEEE 29119 provides guidelines for the implementation of keyword-driven testing. It defines how keywords (or action words) can be used to describe test cases in a way that is independent of the test implementation, enabling better reuse and maintainability of test suites.*
>
> **— ISO/IEC/IEEE 29119-5:2016, Cláusula 1**

Traducción

> *Esta parte de la ISO/IEC/IEEE 29119 proporciona directrices para la implementación de pruebas basadas en palabras clave. Define cómo las palabras clave (o palabras de acción) pueden utilizarse para describir casos de prueba de manera independiente de su implementación, permitiendo una mejor reutilización y mantenibilidad de los conjuntos de pruebas.*
>
> **— ISO/IEC/IEEE 29119-5:2016, Cláusula 1**

La Parte 5 define cómo estructurar pruebas usando palabras clave que describen acciones del usuario de forma independiente de la tecnología. El lenguaje Gherkin (Given/When/Then) es una implementación directa de esta técnica: cada palabra clave describe una acción o verificación sin mencionar código.

App	**Aplicación en este proyecto** En base a la norma ISO/IEC/IEEE 29119-5, los casos de prueba de UI del JD y del Comité se documentaron en formato Gherkin. Por ejemplo, el caso UI-JD-03 usa 'When el JD abre el formulario de una actividad en revisión' y 'Then el tab Alumnos no debe ser visible' — sin mencionar selectores CSS ni código específico de Odoo.

2. ISO/IEC 25010 — Modelo de Calidad del Producto Software

La norma ISO/IEC 25010 forma parte de la familia SQuaRE y define el modelo de calidad que debe evaluarse en un producto de software. Establece ocho características de calidad principales, cada una dividida en subccaracterísticas medibles.

ISO/IEC 25010 ISO / IEC 2011 — rev. 2023	SQuaRE — Modelo de Calidad del Producto

Adecuación Funcional

> *Functional suitability: Degree to which a product or system provides functions that meet stated and implied needs when used under specified conditions. Sub-characteristics: functional completeness, functional correctness, functional appropriateness.*
>
> **— ISO/IEC 25010:2011, Cláusula 4.2.1**

Traducción

> *Adecuación funcional: grado en el que un producto o sistema proporciona funciones que satisfacen necesidades explícitas e implícitas cuando se utiliza bajo condiciones especificadas.*
>
> *Subcaracterísticas: completitud funcional, corrección funcional y adecuación funcional.*
>
> **— ISO/IEC 25010:2011, Cláusula 4.2.1**

La adecuación funcional mide si el sistema hace exactamente lo que el caso de uso especifica: completitud (¿están todas las funciones?), corrección (¿producen los resultados esperados?) y pertinencia (¿son apropiadas para el objetivo?).

App	**Aplicación en este proyecto** En base a la norma ISO/IEC 25010 — Adecuación Funcional, los defectos D-04 (créditos sin rango 1.5 y 2.0), D-13 (campo créditos no resaltado) y D-14 (sin doble confirmación al rechazar) evidencian falta de completitud funcional: el sistema no implementa completamente lo que CA-01SC y JD-01SC especifican.

Seguridad

> *Security: Degree to which a product or system protects information and data so that persons or other products or systems have the degree of data access appropriate to their types and levels of authorization. Sub-characteristics: confidentiality, integrity, non-repudiation, accountability, authenticity.*
> **— ISO/IEC 25010:2011, Cláusula 4.2.6**

Traducción

> *Seguridad: grado en el que un producto o sistema protege la información y los datos, de modo que las personas u otros sistemas tengan el nivel de acceso apropiado según su tipo y nivel de autorización.*
>
> *Subcaracterísticas: confidencialidad, integridad, no repudio, responsabilidad y autenticidad.*
> **— ISO/IEC 25010:2011, Cláusula 4.2.6**

La seguridad evalúa que cada usuario solo pueda acceder y modificar los datos que le corresponden según su rol. La integridad garantiza que los datos no sean modificados por usuarios no autorizados.

App	**Aplicación en este proyecto** En base a la norma ISO/IEC 25010 — Seguridad, el defecto D-08 (JD modifica actividades de otro departamento) es una violación directa de la integridad: un JD puede alterar datos que pertenecen a otro actor sin autorización. La ausencia de ir.rule en el módulo es la causa raíz. Los defectos D-07 y D-15 también comprometen la integridad al permitir modificaciones no autorizadas.

Fiabilidad

> *Reliability: Degree to which a system, product or component performs specified functions under specified conditions for a specified period of time. Sub-characteristics: maturity, availability, fault tolerance, recoverability.*
> **— ISO/IEC 25010:2011, Cláusula 4.2.3**

Traducción

> *Fiabilidad: grado en el que un sistema, producto o componente desempeña funciones especificadas bajo condiciones determinadas durante un periodo de tiempo específico.*
>
> *Subcaracterísticas: madurez, disponibilidad, tolerancia a fallos y capacidad de recuperación.*
> **— ISO/IEC 25010:2011, Cláusula 4.2.3**

La fiabilidad evalúa que el sistema funcione correctamente sin fallar. La tolerancia a fallos mide si el sistema reacciona adecuadamente ante entradas incorrectas, y la madurez evalúa con qué frecuencia falla en condiciones normales.

App	**Aplicación en este proyecto** En base a la norma ISO/IEC 25010 — Fiabilidad, el defecto D-09 (duplicar periodos escolares provoca fallo del módulo) es una violación de tolerancia a fallos: el sistema llega a un estado inestable ante una operación válida del usuario y deja de responder.

Mantenibilidad

> *Maintainability: Degree of effectiveness and efficiency with which a product or system can be modified to improve it, correct it or adapt it to changes in environment, and in requirements. Sub-characteristics: modularity, reusability, analyzability, modifiability, testability.*
>
> **— ISO/IEC 25010:2011, Cláusula 4.2.7**

Traducción

> *Mantenibilidad: grado de efectividad y eficiencia con el que un producto o sistema puede ser modificado para mejorarlo, corregirlo o adaptarlo a cambios en el entorno o en los requisitos.*
>
> *Subcaracterísticas: modularidad, reutilización, analizabilidad, modificabilidad y capacidad de prueba.*
>
> **— ISO/IEC 25010:2011, Cláusula 4.2.7**

La mantenibilidad mide qué tan fácil es modificar el sistema sin introducir errores secundarios. La testeabilidad evalúa si el código puede ser probado de forma efectiva con pruebas automatizadas.

App	**Aplicación en este proyecto** En base a la norma ISO/IEC 25010 — Mantenibilidad/Testeabilidad, el defecto D-00 (envío de recordset en lugar de ID) afecta la testeabilidad: el código funciona en producción gracias a tolerancias del framework, pero falla en pruebas automatizadas. La ejecución de 80 tests Python con cobertura medible demuestra que se aplicó activamente la testeabilidad como criterio de calidad.

3. OCA — Odoo Community Association Guidelines

La Odoo Community Association (OCA) es la organización sin fines de lucro que gobierna el desarrollo de módulos de código abierto para Odoo. Sus directrices establecen los estándares de calidad, estructura y pruebas que deben seguir todos los módulos del ecosistema Odoo Community.

OCA Guidelines Odoo Community Association v19.0	**Estándares de Desarrollo de Módulos Odoo**

Directriz oficial — Estructura de tests

> *All modules must include automated tests. Tests must be placed in a tests/ directory with an __init__.py file. Test classes must inherit from odoo.tests.common.TransactionCase. Each test method must be prefixed with test_. Tests should be tagged with the module name for selective execution using --test-tags.*
>
> **— OCA Development Guidelines — Testing Section, v19.0**

Traducción

> *Todos los módulos deben incluir pruebas automatizadas. Las pruebas deben ubicarse en un directorio tests/ con un archivo __init__.py. Las clases de prueba deben heredar de odoo.tests.common.TransactionCase. Cada método de prueba debe tener el prefijo test_. Las pruebas deben etiquetarse con el nombre del módulo para su ejecución selectiva usando --test-tags.*
>
> **— OCA Development Guidelines — Testing Section, v19.0**

La OCA exige que todo módulo tenga pruebas automatizadas siguiendo la estructura oficial: carpeta tests/ con __init__.py, herencia de TransactionCase, prefijo test_ en cada método y tags para ejecución selectiva.

App	**Aplicación en este proyecto** En base a las OCA Guidelines — Estructura de tests, el módulo actividades_complementarias organiza sus pruebas en tests/ con __init__.py que importa test_propuesta_comite, test_seguridad_comite y test_jefe_departamento. Cada clase hereda de TransactionCase y usa --test-tags actividades_complementarias para ejecución selectiva.

Directriz oficial — Cobertura mínima

> *Coverage of at least 80% of the module's Python code is recommended. Tests should cover: model constraints (@api.constrains), onchange methods (@api.onchange), business logic methods (action_*), and security rules (ir.rule and access control lists).*
>
> **— OCA Development Guidelines — Testing Section, v19.0**

Traducción

> *Se recomienda una cobertura de al menos el 80% del código Python del módulo. Las pruebas deben cubrir: restricciones del modelo (@api.constrains), métodos onchange (@api.onchange), métodos de lógica de negocio (action_*) y reglas de seguridad (ir.rule y listas de control de acceso).*
>
> **— OCA Development Guidelines — Testing Section, v19.0**

La OCA recomienda una cobertura mínima del 80% del código Python y exige que los tests cubran explícitamente: restricciones del modelo, métodos de negocio (action_*) y reglas de seguridad.

App	**Aplicación en este proyecto** En base a las OCA Guidelines — Cobertura, los 80 tests de este proyecto cubren action_aprobar(), action_rechazar(), action_confirmar_aprobacion(), los crons _auto_aprobar_propuestas_vencidas() y _actualizar_estado_por_fecha(), y las restricciones @api.constrains de fechas, cupos y nombre único por periodo.

Directriz oficial — Datos de prueba

> *Demo data for testing should be placed in data/demo_*.xml files declared under the 'demo' key in __manifest__.py — never under 'data'. This ensures demo data is only loaded during development and testing, never in production environments.*
>
> **— OCA Development Guidelines — Demo Data Section, v19.0**

Traducción

> *Los datos de demostración para pruebas deben colocarse en archivos data/demo_*.xml declarados bajo la clave 'demo' en el __manifest__.py, nunca bajo 'data'. Esto garantiza que los datos de prueba solo se carguen durante el desarrollo y las pruebas, nunca en entornos de producción.*
>
> **— OCA Development Guidelines — Demo Data Section, v19.0**

La OCA es explícita: los datos de demostración solo deben declararse bajo 'demo' en el manifest, nunca bajo 'data'. Esto garantiza que usuarios de prueba con contraseñas conocidas nunca lleguen a producción.

App	**Aplicación en este proyecto** En base a las OCA Guidelines — Datos de prueba, el demo_data.xml está correctamente declarado bajo 'demo' en __manifest__.py. Además, los tests crean datos programáticamente en setUpClass() con res.users.create() en lugar de depender del demo data, permitiendo ejecutar los tests en cualquier base de datos.

4. Odoo SA — Directrices Oficiales de la Plataforma

Odoo SA es la empresa creadora y mantenedora de la plataforma Odoo. Sus directrices definen cómo deben construirse los módulos, cómo funciona el framework de pruebas nativo (odoo.tests) y cómo se integra HOOT para pruebas de interfaz de usuario.

Odoo SA Guidelines Odoo SA Odoo 19 — 2024	**Framework de Pruebas Nativo de Odoo**

Directriz oficial — TransactionCase

> *TransactionCase: Each test method runs in its own transaction that is rolled back at the end of the test. This ensures test isolation: changes made in one test do not affect other tests. Use with_user() to test access control and with_context() to inject context variables.*
>
> **— Odoo 19 Developer Documentation — Testing Section**

Traducción

> *TransactionCase: Cada método de prueba se ejecuta en su propia transacción, la cual se revierte (rollback) al finalizar la prueba. Esto asegura el aislamiento de las pruebas: los cambios realizados en una prueba no afectan a las demás. Utilice with_user() para probar el control de acceso y with_context() para inyectar variables de contexto.*
>
> **— Odoo 19 Developer Documentation — Testing Section**

El framework nativo de Odoo 19 garantiza que cada test se ejecute en una transacción aislada que se revierte al finalizar. El método with_user() permite simular sesiones de distintos roles sin cambiar de usuario real.

App	**Aplicación en este proyecto** En base a las directrices de Odoo SA — TransactionCase, los 80 tests Python usan transacciones aisladas. with_user(self.comite_1) simula la sesión del Comité para verificar que no puede crear propuestas (AccessError esperado). with_context(skip_fecha_check=True) inyecta el contexto que el propio módulo provee para fechas pasadas en testing.

Directriz oficial — HOOT (Frontend Testing)

> *HOOT is Odoo's own JavaScript testing framework introduced in Odoo 17, replacing QUnit. Tests are written using describe/test/expect blocks and must be declared in the web.assets_tests bundle in __manifest__.py. HOOT tests run in the browser at /web/tests and interact with Odoo views using @odoo/hoot-dom helpers.*
> **— Odoo 19 Developer Documentation — HOOT Testing Section**

Traducción

> *HOOT es el framework de pruebas de JavaScript propio de Odoo introducido en Odoo 17, reemplazando a QUnit. Las pruebas se escriben usando bloques describe/test/expect y deben declararse en el paquete web.assets_tests en el __manifest__.py. Las pruebas HOOT se ejecutan en el navegador en /web/tests e interactúan con las vistas de Odoo usando los ayudantes de @odoo/hoot-dom.*
> **— Odoo 19 Developer Documentation — HOOT Testing Section**

HOOT es el framework de pruebas de frontend de Odoo 19. Permite escribir pruebas JavaScript que se ejecutan en el navegador e interactúan con las vistas reales de Odoo: formularios, listas, kanbans y wizards. Es el único framework que puede verificar visibilidad de botones, campos editables y decoraciones visuales.

App	**Aplicación en este proyecto** En base a las directrices de Odoo SA — HOOT, los 18 tests HOOT (test_hoot_jd.js) se registran bajo web.assets_tests en __manifest__.py. Verifican defectos de UI no detectables con TransactionCase: que creditos sea readonly en actividades aprobadas (D-06), que el tab Alumnos no aparezca en estado en_revision (D-05) y que los botones correspondan al estado actual.

Directriz oficial — ir.rule y control de acceso por registro

> *Record rules (ir.rule) define row-level access control in Odoo. They are evaluated in addition to access control lists (ir.model.access). A rule defines a domain filter applied automatically to all ORM operations for a given model and group. Without ir.rule, all records of a model are accessible to any user who has the corresponding ACL permission.*
> **— Odoo 19 Developer Documentation — Security — Record Rules**

Traducción

> *Las reglas de registro definen el control de acceso a nivel de fila (registros individuales) en Odoo. Se evalúan además de las listas de control de acceso (ir.model.access). Una regla define un filtro de dominio que se aplica automáticamente a todas las operaciones del ORM para un modelo y grupo determinados. Sin ir.rule, todos los registros de un modelo son accesibles para cualquier usuario que tenga el permiso de ACL correspondiente.*
> **— Odoo 19 Developer Documentation — Security — Record Rules**

Las ir.rule son filtros de dominio que Odoo aplica automáticamente a todas las operaciones ORM. Sin ellas, cualquier usuario con permiso de escritura puede leer y modificar todos los registros del modelo sin restricción por propietario ni departamento.

App	**Aplicación en este proyecto** En base a las directrices de Odoo SA — ir.rule, el defecto D-08 se produce exactamente por su ausencia en actividades_privileges.xml. El ir.model.access.csv otorga perm_write=1 al grupo group_jefe_departamento en actividad.complementaria, pero sin ir.rule que filtre por jefe_departamento_id=user.id, cualquier JD puede editar actividades de otros departamentos.

Directriz oficial — @api.constrains

> *@api.constrains(*field_names): Decorator for constraint methods. The decorated method is called when one of the named fields is set or modified. It should raise odoo.exceptions.ValidationError if the constraint is not satisfied. Constraints are checked by the ORM before writing to the database, providing server-side validation independent of the UI.*
>
> **— Odoo 19 Developer Documentation — ORM — Constraints**

Traducción

> *@api.constrains: Decorador para métodos de restricción. El método decorado se llama cuando uno de los campos nombrados se establece o modifica. Debe lanzar una odoo.exceptions.ValidationError si la restricción no se cumple. El ORM verifica las restricciones antes de escribir en la base de datos, proporcionando una validación del lado del servidor independiente de la interfaz de usuario (UI).*
>
> **— Odoo 19 Developer Documentation — ORM — Constraints**

El decorador @api.constrains garantiza que las validaciones se ejecuten siempre en el servidor, independientemente de si el usuario pasa por la UI o llama a la API JSON-RPC directamente. Es la última línea de defensa para la integridad de los datos.

App	**Aplicación en este proyecto** En base a las directrices de Odoo SA — @api.constrains, los defectos D-02 (horas sin validación), D-04 (créditos sin rango completo) y D-11 (catálogo sin campos obligatorios) corresponden a constraints faltantes. Aunque las vistas tienen required=True, esto solo protege la UI. Si alguien llama action_enviar_catalogo() directamente vía JSON-RPC, el sistema acepta registros incompletos.

5. Mapa de Normativas vs Defectos del Proyecto

La siguiente tabla relaciona cada norma o directriz con los defectos específicos que detectó o motivó en este proyecto.

Norma / Directriz	Característica aplicada	Defectos relacionados
ISO/IEC/IEEE 29119-1	Terminología de defectos/fallos	D-00 a D-19 (clasificación)
ISO/IEC/IEEE 29119-2	Plan de pruebas, criterios de entrada/salida	Plan de pruebas general
ISO/IEC/IEEE 29119-3	Informe de incidencias (estructura)	Todos los defectos documentados
ISO/IEC/IEEE 29119-4	Análisis de valores límite	D-04 (créditos 1.5, 2.0)
ISO/IEC/IEEE 29119-4	Particiones de equivalencia	D-05, D-07, D-10 (estados)
ISO/IEC/IEEE 29119-5	Gherkin / Keyword-driven testing	Casos UI-JD-01 a UI-JD-08
ISO/IEC 25010	Adecuación funcional	D-04, D-13, D-14, D-17
ISO/IEC 25010	Seguridad / Integridad	D-07, D-08, D-15
ISO/IEC 25010	Fiabilidad / Tolerancia a fallos	D-09, D-18
ISO/IEC 25010	Mantenibilidad / Testeabilidad	D-00
OCA Guidelines	Estructura tests/	Todos los tests Python
OCA Guidelines	Cobertura ≥ 80%	80 tests ejecutados
OCA Guidelines	Demo data bajo clave 'demo'	demo_data.xml
Odoo SA — TransactionCase	Aislamiento de tests	80 tests Python
Odoo SA — HOOT	Pruebas de UI en navegador	18 tests HOOT
Odoo SA — ir.rule	Control de acceso por registro	D-08
Odoo SA — @api.constrains	Validación server-side	D-02, D-04, D-11, D-12